

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل یازدهم: Probabilistic information retrieval

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

11.1 Review of basic probability theory

11.2 The Probability Ranking Principle

11.2.1 The 1/0 loss case

11.2.2 The PRP with retrieval costs

11.3 The Binary Independence Model

11.3.1 Deriving a ranking function for query terms

11.3.2 Probability estimates in theory

11.3.3 Probability estimates in practice

11.3.4 Probabilistic approaches to relevance feedback

11.4 An appraisal and some extensions

11.4.1 An appraisal of probabilistic models

11.4.2 Tree-structured dependencies between terms

11.4.3 Okapi BM25: a non-binary model

11.4.4 Bayesian network approaches to IR

11.5 Chapter 11 Summary

فصل ۱۱: بازیابی اطلاعات احتمالاتی

در فصل‌های قبل، مدل‌های Boolean و فضای برداری برای بازیابی اطلاعات معرفی شدند. این مدل‌ها مانند یک کلیداند که فقط در صورت تطابق کامل قفل را باز می‌کنند، اما در دنیای واقعی، جستجو اغلب مبهم و ناقص است. در این فصل، رویکردی متفاوت و اصولی بر پایهٔ **نظریهٔ احتمال** ارائه می‌شود که به سیستم اجازه می‌دهد تحت **عدم قطعیت** تصمیم‌گیری کند.

مثال واقعی: استفادهٔ آموزشی از IBM Watson در تشخیص پزشکی

در بیمارستان‌های آموزشی، دانشجویهای پزشکی هنگام کار با بیماران واقعی از Watson به عنوان یک ابزار «تصمیم‌یار» استفاده می‌کردند. هنگامی که دانشجوی علائمی مثل تنگی نفس، سرفه خشک یا کاهش اکسیژن خون را وارد می‌کرد، Watson فقط کلمات را مطابقت نمی‌داد، بلکه با توجه به داده‌های میلیون‌ها پروندهٔ قبلی و مدل‌های آماری خود، احتمال هر بیماری را محاسبه می‌کرد. برای مثال، اگر بیمار سرفه خشک، تنگی نفس و افت اکسیژن داشت، Watson احتمال حملهٔ آسم را بالاتر از آنفولانزا قرار می‌داد و گزینه‌هایی مثل COVID-19 یا ذات‌الریه را نیز با درصدی مشخص نمایش می‌داد. این تحلیل احتمالاتی به دانشجوی کمک می‌کرد بفهمد کدام تشخیص‌ها «محتمل‌تر» هستند و چه آزمایش‌های تکمیلی منطقی‌تر است.

در مدل احتمالاتی، اگر اسناد مرتبط و نامرتبتي در اختیار داشته باشیم، می‌توانیم احتمال ظاهر شدن یک واژه در سند مرتبط را تخمین بزنیم:

$$P(t | R = 1)$$

این مقدار می‌تواند پایه‌ای برای ساخت یک **طبقه‌بند** باشد که تصمیم می‌گیرد آیا سندی مرتبط است یا نه. در این فصل، این رویکرد به صورت سیستماتیک معرفی می‌شود.

مروری بر نظریه پایه‌ی احتمال

نظریه احتمال به ما کمک می‌کند تا در دنیای عدم قطعیت، تصمیمات منطقی بگیریم. متغیر تصادفی A نمایانگر یک رویداد است. احتمال وقوع آن بین ۰ تا ۱ است:

$$0 \leq P(A) \leq 1$$

مثال: فیلتر اسپم در جیمیل

جیمیل از نظریه احتمال برای فیلتر کردن ایمیل‌های اسپم استفاده می‌کند. برای هر ایمیل، سیستم احتمال اینکه آن اسپم باشد را محاسبه می‌کند. اگر ایمیل حاوی کلماتی مانند «ارائه ویژه» یا «برنده» باشد، احتمال اسپم بودن آن افزایش می‌یابد. این احتمال با در نظر گرفتن ویژگی‌های دیگر مانند فرستنده و تاریخچه ایمیل دقیق‌تر می‌شود.

برای دو رویداد A و B، احتمال مشترک آن‌ها با قانون زنجیره‌ای (Chain Rule) تعریف می‌شود:

$$P(A, B) = P(A \cap B) = P(A | B) \cdot P(B) = P(B | A) \cdot P(A)$$

قانون تقسیم (Partition Rule) بیان می‌کند که اگر B به زیرمجموعه‌های مجزا تقسیم شود، آنگاه:

$$P(B) = P(A, B) + P(\neg A, B)$$

اصل رتبه‌بندی احتمالاتی

اصل رتبه‌بندی احتمالاتی (Probability Ranking Principle) بیان می‌کند که اسناد باید بر اساس احتمال مرتبط بودن رتبه‌بندی شوند. این اصل، پایه بسیاری از مدل‌های بازیابی اطلاعات مدرن است.

مثال: الگوریتم PageRank گوگل

الگوریتم PageRank که گوگل در ابتدا از آن استفاده می‌کرد، بر اساس اصل رتبه‌بندی احتمالاتی عمل می‌کند. PageRank احتمال اینکه کاربر با کلیک روی یک لینک به صفحه‌ای خاص برسد را محاسبه می‌کند و صفحات را بر اساس این احتمال رتبه‌بندی می‌کند. صفحاتی که احتمال بازدید بیشتری دارند، در رتبه‌های بالاتر قرار می‌گیرند.

قانون بیز (Bayes' Rule): برای معکوس کردن احتمال شرطی استفاده می‌شود:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)} = \frac{P(B | A) \cdot P(A)}{\sum_{X \in \{A, \neg A\}} P(B | X) \cdot P(X)}$$

مثال: سیستم توصیه نتفلیکس

نتفلیکس از قانون بیز برای توصیه فیلم استفاده می‌کند. اگر کاربری فیلم‌های اکشن را دوست داشته باشد (احتمال اولیه)، نتفلیکس با توجه به اینکه کاربر اخیراً به فیلم‌های ابرقهرمانی علاقه نشان داده (شواهد)، احتمال پس از مشاهده شواهد را به‌روز می‌کند. این رویکرد به نتفلیکس اجازه می‌دهد توصیه‌های دقیق‌تری به کاربر ارائه دهد.

در این فرمول:

- $P(A)$: احتمال اولیه (prior)
- $P(A|B)$: احتمال پس از مشاهده شواهد (posterior)
- $P(B|A)$: احتمال مشاهده شواهد در صورت صحت فرض

مدل استقلال دودویی

مدل استقلال دودویی (Binary Independence Model) یکی از اولین و تأثیرگذارترین مدل‌های بازیابی احتمالاتی است. این مدل فرض می‌کند که واژه‌ها در اسناد به صورت مستقل از هم ظاهر می‌شوند.

مثال: موتور جستجوی علمی PubMed

PubMed که برای جستجوی مقالات علمی پزشکی استفاده می‌شود، از مدل استقلال دودویی برای رتبه‌بندی نتایج استفاده می‌کند. این مدل فرض می‌کند که ظاهر شدن واژه‌ها در یک مقاله، مستقل از ظاهر شدن واژه‌های دیگر است. اگرچه این فرض ساده‌شده است، اما در عمل نتایج قابل قبولی ارائه می‌دهد و پایه بسیاری از سیستم‌های پیشرفته‌تر است.

در فصل‌های بعدی، مدل‌های پیشرفته‌تری مانند Okapi BM25 و مدل‌های شبکه بیزی معرفی خواهند شد که این فرض ساده را بهبود می‌بخشند.

۱۱.۲.۱ حالت Loss 1/0 در اصل رتبه‌بندی احتمالاتی (PRP)

در این بخش، فرض می‌کنیم که سیستم بازیابی اطلاعات در یک چارچوب رتبه‌دار عمل می‌کند (مطابق با فصل ۶.۳): کاربر یک پرسمان ارسال می‌کند و سیستم فهرستی مرتب‌شده از اسناد را باز می‌گرداند. همچنین فرض می‌کنیم که مرتبط بودن به صورت دودویی تعریف شده است (مطابق فصل ۸): هر سند یا مرتبط است یا نیست.

مثال واقعی: سیستم اولویت‌بندی ایمیل در Gmail

در سرویس Gmail، بخش «Important» بر پایه یک مدل احتمالاتی عمل می‌کند. سیستم برای هر ایمیل ورودی بررسی می‌کند که چقدر احتمال دارد این ایمیل برای کاربر مهم باشد: این احتمال از ترکیبی از داده‌هایی مثل فرستنده، سابقه پاسخ‌گویی کاربر، کلمات کلیدی، و رفتارهای گذشته محاسبه می‌شود. سپس ایمیل‌ها بر اساس همین احتمال اهمیت رتبه‌بندی می‌شوند. در این مدل، هر ایمیل عملاً برچسبی دودویی دارد (مهم یا غیرمهم) و Gmail فهرست را طوری مرتب می‌کند که ایمیل‌هایی با احتمال «مهم بودن» بالاتر، در بالاترین رتبه‌ها نمایش داده شوند.

برای هر پرسمان q و سند d ، متغیر تصادفی $R_{d,q}$ نشان می‌دهد که آیا سند d نسبت به پرسمان q مرتبط است یا نه:

$$R_{d,q} = \begin{cases} 1 & \text{if document } d \text{ is relevant to query } q \\ 0 & \text{otherwise} \end{cases}$$

در این مدل، طبیعی‌ترین روش برای رتبه‌بندی اسناد، مرتب‌سازی آن‌ها بر اساس احتمال مرتبط بودن است:

$$P(R = 1 \mid d, q)$$

این ایده، پایه اصل رتبه‌بندی احتمالاتی (Probability Ranking Principle) است که توسط van Rijsbergen (1979) معرفی شد:

اگر یک سیستم بازیابی اطلاعات، در پاسخ به هر پرسمان، اسناد را به ترتیب احتمال نزولی مرتبط بودن با نیاز اطلاعاتی کاربر مرتب کند – به طوری که این احتمال‌ها تا حد ممکن دقیق تخمین زده شده باشند – آنگاه کارایی کلی سیستم برای کاربر، در مقایسه با هر روش دیگری که فقط از همان داده‌ها استفاده کند، پیشینه خواهد بود.

در ساده‌ترین حالت، هیچ هزینه‌ای برای بازیابی یا سودمندی خاصی در نظر گرفته نمی‌شود. فقط دو نوع خطا وجود دارد:

- بازگرداندن سند نامرتب
- عدم بازگرداندن سند مرتبط

این وضعیت به عنوان **Loss 1/0** شناخته می‌شود: هر خطا یک واحد هزینه دارد. هدف، بازگرداندن بهترین اسناد ممکن در میان k سند اول است – برای هر مقداری از k که کاربر بررسی می‌کند.

مثال: سیستم اولویت‌بندی اخبار در اپلیکیشن‌های خبری

بسیاری از اپلیکیشن‌های خبری (مانند Google News یا Apple News) از مدل‌های احتمالاتی برای رتبه‌بندی اخبار استفاده می‌کنند. برای هر خبر، احتمال اینکه کاربر به آن علاقه‌مند باشد محاسبه می‌شود. این احتمال بر اساس سابقه مطالعه کاربر، موضوعات مورد علاقه، و اخبار مشابهی که قبلاً خوانده، تعیین می‌گردد. سپس اخبار با احتمال بالاتر در بالای فید کاربر نمایش داده می‌شوند. این سیستم بر اساس $Loss\ 1/0$ عمل می‌کند: نشان دادن یک خبر نامرتب در بالا (False Positive) ممکن است باعث از دست رفتن علاقه کاربر شود، اما عدم نمایش یک خبر بسیار مهم (False Negative) هزینه بیشتری دارد. هدف، کمینه کردن این خطاها و ارائه بهترین خبرها در ابتدای فهرست است.

در این حالت، PRP می‌گوید که باید اسناد را به ترتیب نزولی $P(R = 1 | d, q)$ مرتب کنیم.

اگر به جای رتبه‌بندی، بخواهیم مجموعه‌ای از اسناد را بازگردانیم، **قانون تصمیم بهینه بیزی (Bayes Optimal Decision Rule)** می‌گوید:

Doc d It is relevant if and only if $P(R = 1 | d, q) > P(R = 0 | d, q)$

این قانون تصمیم، **ریسک مورد انتظار** را در حالت $Loss\ 1/0$ کمینه می‌کند.

مثال: سیستم تشخیص تقلب در تراکنش‌های بانکی

بانک‌ها برای تشخیص تراکنش‌های جعلی از مدل‌های احتمالاتی پیچیده استفاده می‌کنند. سیستم برای هر تراکنش، احتمال اینکه آن تراکنش «تقلب‌آمیز» باشد را محاسبه می‌کند. اگر این احتمال از یک آستانه مشخص (که بر اساس تحلیل هزینه خطاها تعیین شده) بالاتر باشد، تراکنش مسدود شده یا برای تأیید بیشتر علامت‌گذاری می‌شود. این استفاده مستقیم از قانون تصمیم بیزی (Bayes Optimal Decision Rule) است؛ جایی که هزینه رد کردن یک تراکنش صحیح (False Positive) و هزینه پذیرش یک تراکنش جعلی (False Negative) به دقت محاسبه و در تصمیم‌گیری لحاظ می‌شود تا ریسک کلی به حداقل برسد.

قضیه ۱۱.۱: اصل رتبه‌بندی احتمالاتی (PRP) در حالت $Loss\ 1/0$ بهینه است، به این معنا که **ریسک مورد انتظار** را کمینه می‌کند.

اثبات این قضیه در Ripley (1996) آمده است، اما فرض می‌کند که همه احتمالات به درستی شناخته شده‌اند – که در عمل هرگز صادق نیست. با این حال، PRP همچنان پایه‌ای بسیار مفید برای طراحی مدل‌های بازیابی اطلاعات محسوب می‌شود.

۱۱.۲.۲ اصل رتبه‌بندی احتمالاتی با هزینه‌های بازیابی

در حالت ساده PRP، فرض می‌شود که همه خطاها هزینه یکسانی دارند ($Loss\ 1/0$). اما در کاربردهای واقعی، هزینه بازگرداندن سند نامرتب با هزینه عدم بازگرداندن سند مرتبط یکسان نیست. در این بخش، مدل رتبه‌بندی با در نظر گرفتن **هزینه‌های بازیابی** معرفی می‌شود.

مثال واقعی: سیستم پیشنهاد محصول در فروشگاه‌های آنلاین

در فروشگاه‌های آنلاین مانند دیجی‌کالا، نمایش یک محصول نامرتب به کاربر (False Positive) ممکن است باعث اتلاف وقت و کاهش احتمال خرید شود، اما عدم نمایش یک محصول بسیار مرتبط که کاربر به دنبال آن است (False Negative) می‌تواند منجر به از دست دادن یک فروش قطعی شود. بنابراین، سیستم‌های پیشنهاد محصول طوری طراحی می‌شوند که هزینه خطای False Negative (از دست دادن فروش) را بسیار بیشتر از False Positive در نظر بگیرند تا شانس موفقیت در فروش را به حداکثر برسانند.

فرض کنیم:

- C_1 : هزینه عدم بازیابی یک سند مرتبط (false negative)
- C_0 : هزینه بازیابی یک سند نامرتب (false positive)

در این مدل، PRP می‌گوید سندی را باید انتخاب کنیم که **ریسک مورد انتظار** آن کمینه باشد. برای سند d ، اگر برای همه سندهای d' که هنوز بازیابی نشده‌اند، رابطه زیر برقرار باشد، آنگاه d باید سند بعدی باشد که بازیابی می‌شود:

$$C_0 \cdot P(R = 0 | d) - C_1 \cdot P(R = 1 | d) \leq C_0 \cdot P(R = 0 | d') - C_1 \cdot P(R = 1 | d')$$

این مدل یک چارچوب رسمی برای مدل‌سازی هزینه‌های متفاوت خطاها فراهم می‌کند – برخلاف فصل ۸.۶ که فقط در مرحله ارزیابی به این موضوع پرداختیم، اینجا در **مرحله مدل‌سازی** لحاظ می‌شود.

مثال: سیستم اولویت‌بندی وظایف در نرم‌افزارهای مدیریت پروژه

در نرم‌افزارهای مدیریت پروژه مانند Asana یا Trello، گاهی لازم است بین وظایف مختلف اولویت‌بندی صورت گیرد. عدم پیگیری یک وظیفه حیاتی (False Negative) می‌تواند منجر به تأخیر در پروژه و هزینه‌های هنگفتی شود، در حالی که صرف زمان اندکی برای بررسی یک وظیفه کم‌اهمیت (False Positive) ممکن است هزینه کمتری داشته باشد. بنابراین، سیستم اولویت‌بندی، وظایف را بر اساس ترکیبی از فوریت و اهمیت، و با در نظر گرفتن هزینه بالاتر خطاهای False Negative، مرتب می‌کند تا اطمینان حاصل شود که مهم‌ترین کارها در اولویت اجرا قرار گیرند.

با این حال، در ادامه فصل، مدل‌های سود/زیان پیچیده‌تر بررسی نمی‌شوند. اما این بخش نشان می‌دهد که PRP می‌تواند به صورت تعمیم‌یافته برای شرایط واقعی‌تر نیز استفاده شود.

۱۱.۳ مدل دودویی استقلال (Binary Independence Model)

مدل دودویی استقلال (BIM) یکی از مدل‌های کلاسیک بازیابی اطلاعات است که با اصل رتبه‌بندی احتمالاتی (PRP) ترکیب می‌شود. این مدل با فرض‌های ساده‌ای طراحی شده که محاسبه احتمال مرتبط بودن سند را عملی و قابل پیاده‌سازی می‌کند.

مثال واقعی: سیستم پیشنهاد فیلم در سرویس‌های استریم

سرویس‌های استریم مانند نتفلیکس یا فیلیمو برای پیشنهاد فیلم و سریال به کاربران از مدل‌های مشابه دودویی استقلال استفاده می‌کنند. این سیستم‌ها با تحلیل حضور یا عدم حضور کلمات کلیدی در توضیحات فیلم، ژانرها، بازیگران و کارگردانان، احتمال مرتبط بودن یک فیلم با علائق کاربر را تخمین می‌زنند. به عنوان مثال، اگر کاربر به دنبال فیلم‌های “علمی-تخیلی” و “اکشن” باشد، سیستم فیلم‌هایی را که هر دو کلمه کلیدی را در توضیحات خود دارند، در اولویت نمایش قرار می‌دهد. این مدل، اگرچه فرض استقلال بین کلمات را دارد، اما در عمل برای ارائه پیشنهادهاى اولیه بسیار کارآمد است.

در این مدل:

- نمایش سند و پرسمان به صورت بردار دودویی از واژه‌ها است: اگر واژه t در سند d وجود داشته باشد، $x_t = 1$ و در غیر این صورت $x_t = 0$.
- نمایش پرسمان نیز به صورت بردار وقوع واژه‌ها است: $\vec{q}$
- فرض استقلال: وقوع واژه‌ها در سندها مستقل از یکدیگر فرض می‌شود (مانند فرض ساده‌انگارانه Naive Bayes).

این فرض‌ها در عمل نادرست‌اند (مثلاً واژه‌ها هم‌وابسته‌اند یا اسناد تکراری وجود دارند)، اما در بسیاری از کاربردها نتایج قابل قبولی تولید می‌کنند.

هدف مدل، تخمین احتمال مرتبط بودن سند d با پرسمان q است، یعنی:

$$P(R = 1 \mid \vec{x}, \vec{q})$$

با استفاده از قانون بیز، این مقدار به صورت زیر محاسبه می‌شود:

$$P(R = 1 \mid \vec{x}, \vec{q}) = \frac{P(\vec{x} \mid R = 1, \vec{q}) \cdot P(R = 1 \mid \vec{q})}{P(\vec{x} \mid \vec{q})}$$

$$P(R = 0 \mid \vec{x}, \vec{q}) = \frac{P(\vec{x} \mid R = 0, \vec{q}) \cdot P(R = 0 \mid \vec{q})}{P(\vec{x} \mid \vec{q})}$$

در این فرمول‌ها:

- $P(\vec{x} \mid R = 1, \vec{q})$: احتمال مشاهده بردار سند در میان اسناد مرتبط
- $P(\vec{x} \mid R = 0, \vec{q})$: احتمال مشاهده بردار سند در میان اسناد نامرتب
- $P(R = 1 \mid \vec{q})$: احتمال اولیه مرتبط بودن سند با پرسمان
- $P(R = 0 \mid \vec{q})$: احتمال اولیه نامرتب بودن سند با پرسمان

چون سند یا مرتبط است یا نیست، باید داشته باشیم:

$$P(R = 1 \mid \vec{x}, \vec{q}) + P(R = 0 \mid \vec{x}, \vec{q}) = 1$$

در عمل، هیچ‌کدام از این احتمال‌ها به صورت دقیق قابل محاسبه نیستند. بنابراین، از **آمار مجموعه اسناد** برای تخمین آن‌ها استفاده می‌شود. این شامل:

- فراوانی واژه در سند (term frequency)
- فراوانی واژه در مجموعه (document frequency)
- طول سند

مثال: سیستم جستجو در فروشگاه‌های آنلاین محصولات تخصصی

فروشگاه‌های آنلاینی که محصولات تخصصی مانند قطعات الکترونیکی یا ابزارهای صنعتی را عرضه می‌کنند، اغلب از مدل دودویی استقلال برای سیستم جستجوی خود بهره می‌برند. در این سیستم‌ها، حضور یا عدم حضور واژه‌های کلیدی در نام محصول، توضیحات فنی، یا مشخصات آن، مبنای رتبه‌بندی نتایج قرار می‌گیرد. به عنوان مثال، اگر کاربری به دنبال "پردازنده Core i7 با فرکانس 3.5 گیگاهرتز" باشد، سیستم به حضور دقیق این عبارات در توضیحات محصول اولویت می‌دهد. اگرچه ممکن است ترکیب این واژه‌ها در محصولات مختلف مستقل نباشد (مثلاً "Core i7" و "گیگاهرتز" اغلب با هم می‌آیند)، اما این مدل به دلیل سادگی پیاده‌سازی و کارایی قابل قبول، همچنان در بسیاری از این فروشگاه‌ها استفاده می‌شود.

در نهایت، اسناد بر اساس احتمال تخمینی مرتبط بودن مرتب می‌شوند.

۱۱.۳.۱ استخراج تابع رتبه‌بندی برای واژه‌های پرسمان

در مدل دودویی استقلال، هدف رتبه‌بندی اسناد بر اساس احتمال مرتبط بودن آن‌ها با پرسمان است. اما چون هدف فقط رتبه‌بندی است (نه مقدار دقیق احتمال)، می‌توان از **نسبت شانس (odds)** استفاده کرد که با احتمال مرتبط بودن هم‌مرتبه (monotonic) است.

مثال واقعی: سیستم جستجوی علمی PubMed

PubMed برای رتبه‌بندی مقالات علمی از یک مدل احتمالاتی استفاده می‌کند. وقتی شما عبارت "CRISPR-Cas9" و قلب عروقی" را جستجو می‌کنید، سیستم احتمال وقوع هر واژه در مقالات مرتبط را محاسبه می‌کند. سپس مقالات را بر اساس نسبت شانس (odds) رتبه‌بندی می‌کند. واژه‌هایی که در مقالات مرتبط شانس بالاتری برای وقوع دارند، در رتبه‌های بالاتر قرار می‌گیرند.

در مدل BIM، نسبت شانس وقوع واژه در سند مرتبط به نامرتب به صورت زیر محاسبه می‌شود:

$$O(R | \vec{x}, \vec{q}) = \frac{P(R = 1 | \vec{x}, \vec{q})}{P(R = 0 | \vec{x}, \vec{q})}$$

با استفاده از قانون بیز و حذف مخرج مشترک، داریم:

$$O(R | \vec{x}, \vec{q}) = \frac{P(R = 1 | \vec{x}, \vec{q})}{P(R = 0 | \vec{x}, \vec{q})} \cdot \frac{P(\vec{x} | R = 1, \vec{q})}{P(\vec{x} | R = 0, \vec{q})}$$

عبارت اول برای هر پرسمان ثابت است و در رتبه‌بندی نهایی تأثیری ندارد. بنابراین، فقط باید عبارت دوم را تخمین بزنیم. برای این کار، فرض استقلال شرطی واژه‌ها (Naive Bayes) را اعمال می‌کنیم:

$$\frac{P(\vec{x} | R = 1, \vec{q})}{P(\vec{x} | R = 0, \vec{q})} = \prod_{t=1}^M \frac{P(x_t | R = 1, \vec{q})}{P(x_t | R = 0, \vec{q})}$$

چون x_t فقط ۰ یا ۱ است، می‌توان حاصل ضرب را به دو بخش تقسیم کرد:

$$O(R | \vec{x}, \vec{q}) = O(R | \vec{q}) \cdot \prod_{t:x_t=1} \frac{p_t}{u_t} \cdot \prod_{t:x_t=0} \frac{1-p_t}{1-u_t}$$

که در آن:

- $p_t = P(x_t = 1 | R = 1, q)$: احتمال وقوع واژه در سند مرتبط
- $u_t = P(x_t = 1 | R = 0, q)$: احتمال وقوع واژه در سند نامرتب

با فرض ساده‌ساز دیگر: واژه‌هایی که در پرسمان نیستند، احتمال وقوعشان در اسناد مرتبط و نامرتب برابر است (یعنی $p_t = u_t$ برای $q_t = 0$). بنابراین فقط واژه‌های موجود در پرسمان در رتبه‌بندی مؤثرند:

$$O(R | \vec{x}, \vec{q}) = O(R | \vec{q}) \cdot \prod_{t:x_t=q_t=1} \frac{p_t}{u_t} \cdot \prod_{t:x_t=0, q_t=1} \frac{1-p_t}{1-u_t}$$

با بازنویسی جبری، حاصل ضرب دوم را به حاصل ضرب اول منتقل می‌کنیم و به فرم نهایی می‌رسیم:

$$O(R | \vec{x}, \vec{q}) = \text{const} \cdot \prod_{t:x_t=q_t=1} \frac{p_t(1-u_t)}{u_t(1-p_t)}$$

چون تابع لگاریتم یکنواخت است، می‌توان به جای حاصل ضرب، مجموع لگاریتم‌ها را استفاده کرد. این مقدار به عنوان **Retrieval Status Value (RSV)** شناخته می‌شود:

$$RSV_d = \sum_{t:x_t=q_t=1} \log \left(\frac{p_t(1-u_t)}{u_t(1-p_t)} \right)$$

برای هر واژه پرسمان، وزن c_t تعریف می‌شود:

$$c_t = \log \left(\frac{p_t}{1-p_t} \right) + \log \left(\frac{1-u_t}{u_t} \right)$$

این وزن‌ها نشان می‌دهند که واژه چقدر بیشتر در اسناد مرتبط نسبت به نامرتب ظاهر می‌شود. اگر $p_t = u_t$ باشد، آنگاه $c_t = 0$ و واژه در رتبه‌بندی تأثیری ندارد.

در عمل، برای هر سند، مجموع وزن‌های واژه‌های پرسمان موجود در آن محاسبه می‌شود:

$$RSV_d = \sum_{t:x_t=q_t=1} c_t$$

مثال: سیستم جستجوی حقوقی Westlaw

Westlaw که برای جستجوی قوانین حقوقی استفاده می‌شود، از یک مدل احتمالاتی برای رتبه‌بندی نتایج استفاده می‌کند. وقتی یک وکیل به دنبال "قانون حفظ حریم خصوصی در قراردادهای می‌گردد، سیستم احتمال وقوع هر واژه در اسناد حقوقی مرتبط را محاسبه می‌کند. سپس اسناد را بر اساس RSV رتبه‌بندی می‌کند. این رویکرد باعث می‌شود اسنادی که واژه‌های کلیدی بیشتری دارند، در رتبه‌های بالاتر قرار گیرند.

در نهایت، اسناد بر اساس RSV رتبه‌بندی می‌شوند که یک رویکرد عملی برای رتبه‌بندی در مدل دودویی استقلال است.

۱۱.۳.۲ تخمین‌های احتمالی در نظریه

در این بخش، هدف محاسبهٔ وزن c_t برای هر واژه t در مجموعهٔ اسناد است. این وزن‌ها نشان می‌دهند که واژه چقدر بیشتر در اسناد مرتبط نسبت به نامرتب‌ظاهر می‌شود. برای این کار، از جدول شمارش زیر استفاده می‌کنیم:

Term	Relevant	Nonrelevant	Total
Present ($x_t = 1$)	s	$dft - s$	dft
Absent ($x_t = 0$)	$S - s$	$(N - dft) - (S - s)$	$N - dft$
Total	S	$N - S$	N

در این جدول:

- N : تعداد کل اسناد
- S : تعداد اسناد مرتبط
- dft : تعداد اسنادی که واژهٔ t را دارند
- s : تعداد اسناد مرتبطی که واژهٔ t را دارند

با استفاده از این جدول، احتمال‌های شرطی به‌صورت زیر تعریف می‌شوند:

$$p_t = \frac{s}{S}, \quad u_t = \frac{dft - s}{N - S}$$

و وزن c_t به‌صورت زیر محاسبه می‌شود:

$$c_t = \log \left(\frac{s/(S - s)}{(dft - s)/((N - dft) - (S - s))} \right)$$

اما این فرمول ممکن است در صورت صفر بودن صورت یا مخرج دچار شکست شود. برای جلوگیری از این مشکل، از **نرم‌سازی (smoothing)** استفاده می‌کنیم – به‌ویژه با افزودن $1/2$ به شمارش‌ها:

$$\hat{c}_t = \log \left(\frac{(s + 1/2)/(S - s + 1/2)}{(dft - s + 1/2)/(N - dft - S + 1/2)} \right)$$

این روش نرم‌سازی، یک فرم ساده از **تخمین بیشینهٔ پسین (MAP)** است – که در آن، فرض اولیهٔ یکنواخت ضعیف داریم و سپس بر اساس داده‌های مشاهده‌شده، احتمال‌ها را به‌روزرسانی می‌کنیم.

در تخمین‌های احتمالی برای داده‌های دسته‌ای (مثل حضور یا غیاب واژه)، رایج‌ترین روش تخمین، **فراوانی نسبی (Relative Frequency)** است:

$$P(\text{event}) = \frac{\text{occured number}}{\text{number of all experiment}}$$

این مقدار، **تخمین بیشینهٔ درست‌نمایی (MLE)** است – یعنی مقداری که داده‌های مشاهده‌شده را بیشینه می‌کند. اما اگر فقط از MLE استفاده کنیم:

- احتمال واژه‌های دیده‌شده بیش از حد بالا تخمین زده می‌شود
- واژه‌های دیده‌نشده احتمال صفر می‌گیرند، که مدل را می‌شکند

برای حل این مشکل، از **نرم‌سازی با شبه‌شمارش (pseudocounts)** استفاده می‌کنیم – مثلاً افزودن $\alpha = 1/2$ به هر شمارش. این معادل با فرض اولیهٔ بیزی یکنواخت است.

در فصل ۱۲.۲.۲ روش‌های پیشرفته‌تر نرم‌سازی بررسی خواهند شد، اما در حال حاضر، افزودن $1/2$ کفایت می‌کند.

مثال: سیستم جستجوی علمی PubMed

PubMed برای رتبه‌بندی مقالات علمی از یک مدل احتمالاتی استفاده می‌کند. وقتی شما عبارت "CRISPR-Cas9" و قلب عروقی را جستجو می‌کنید، سیستم احتمال وقوع هر واژه در مقالات مرتبط را محاسبه می‌کند. برای تخمین این احتمال‌ها، از فراوانی نسبی واژه‌ها در کل مجموعه مقالات استفاده می‌کند. این رویکرد باعث می‌شود واژه‌های نادر مانند "CRISPR" که در مقالات نادر رایج نیستند، احتمال پایینی بگیرند و در رتبه‌بندی تأثیر منفی داشته باشند.

در نهایت، این روش‌های تخمین احتمال، پایهٔ مدل‌های بازیابی احتمالاتی مانند BIM را تشکیل می‌دهند و به سیستم اجازه می‌دهند تا با عدم قطعیت، تصمیمات منطقی بگیرد.

۱۱.۳.۳ تخمین‌های احتمالی در عمل

در عمل، معمولاً فرض می‌شود که اسناد مرتبط درصد بسیار کوچکی از کل مجموعه هستند. بنابراین، می‌توان آمار اسناد نامرتب را با آمار کل مجموعه تقریب زد. تحت این فرض، احتمال وقوع واژه t در اسناد نامرتب (یعنی u_t) به‌صورت زیر تخمین زده می‌شود:

$$u_t = \frac{df_t}{N}$$

و در نتیجه، عبارت لگاریتمی زیر که در وزن‌دهی RSV ظاهر می‌شود، به فرم ساده‌ای تبدیل می‌شود:

$$\log\left(\frac{1 - u_t}{u_t}\right) = \log\left(\frac{N - df_t}{df_t}\right) \approx \log\left(\frac{N}{df_t}\right)$$

این تقریب، توجیه نظری برای وزن‌دهی **IDF** است که در فصل ۶.۲.۱ معرفی شد.

اما تخمین p_t (احتمال وقوع واژه در اسناد مرتبط) به‌سادگی قابل تقریب نیست. سه روش برای تخمین آن وجود دارد:

۱. **استفاده از اسناد مرتبط شناخته‌شده:** اگر تعدادی سند مرتبط داشته باشیم، می‌توانیم p_t را مستقیماً از

آن‌ها تخمین بزنیم. این پایهٔ روش‌های بازخورد مرتبط بودن (relevance feedback) است که در بخش بعدی بررسی می‌شود.

۲. **مدل Croft & Harper (1979):** فرض می‌شود که $p_t = 0.5$ برای همهٔ واژه‌های پرسمان. در این حالت،

چون $p_t = 1 - p_t$ ، عوامل مربوط به p_t در فرمول RSV حذف می‌شوند و فقط IDF باقی می‌ماند.

این روش برای اسناد کوتاه (مثل عنوان یا چکیده) و جست‌وجوی غیرتکراری قابل قبول است.

۳. **مدل Greiff (1998):** نشان داد که p_t در عمل با df_t افزایش می‌یابد. پیشنهاد او برای تخمین p_t

به‌صورت زیر است:

$$p_t = \frac{1}{3} + \frac{2}{3} \cdot \frac{df_t}{N}$$

روش‌های ترکیبی و تکراری برای تخمین p_t در بخش بعدی بررسی خواهند شد. اما تا اینجا، می‌توان گفت که در غیاب داده‌های مرتبط، استفاده از IDF به‌عنوان تقریب برای وزن‌دهی واژه‌ها در مدل BIM قابل توجیه است. در نهایت، این روش‌های تخمین احتمال، پایه‌ی مدل‌های بازیابی احتمالاتی مانند BIM را تشکیل می‌دهند و به سیستم اجازه می‌دهند تا با عدم قطعیت، تصمیمات منطقی بگیرد.

۱۱.۳.۴ رویکردهای احتمالاتی به بازخورد مرتبط بودن

برای تخمین دقیق‌تر p_t ، می‌توان از بازخورد مرتبط بودن (واقعی یا شبه) استفاده کرد. الگوریتم بازخورد احتمالاتی به‌صورت زیر عمل می‌کند:

- تخمین اولیه p_t و u_t با فرض‌های ساده (مثلاً $p_t = 0.5$)
- بازیابی اسناد بر اساس تخمین‌های فعلی و ارائه آن‌ها به کاربر
- دریافت قضاوت‌های مرتبط بودن از کاربر برای مجموعه‌ای از اسناد V
- به‌روزرسانی تخمین‌ها با استفاده از داده‌های مرتبط
- تکرار مراحل تا زمانی که رتبه‌بندی به حالت پایدار برسد

در تخمین اولیه، فرض می‌کنیم که کاربر تقریباً ۵٪ از اسناد را مرتبط تشخیص می‌دهد ($\kappa = 0.5$). در این حالت، تخمین اولیه برای p_t و u_t به‌صورت زیر خواهد بود:

$$p_t^{(0)} = 0.5, \quad u_t^{(0)} = 0.5$$

در مرحله بازیابی، مجموعه‌ای از اسناد با امتیاز بالا (مثلاً ۱۰ سند برتر) انتخاب می‌شود. سپس این مجموعه به کاربر ارائه می‌شود و کاربر قضاوت خود را برای برخی از این اسناد ثبت می‌کند.

پس از دریافت قضاوت‌های مرتبط، تخمین‌های جدید به‌صورت زیر محاسبه می‌شوند:

$$p_t^{(k+1)} = \frac{|V_R^t| + \kappa p_t^{(k)}}{|V_R| + \kappa}$$

که در آن:

- $|V_R^t|$: تعداد اسناد مرتبطی که واژه t را دارند
- $|V_R|$: تعداد کل اسناد مرتبطی که قضاوت برای آن‌ها دریافت شده
- κ : ضریبی که نشان می‌دهد چقدر به قضاوت‌های کاربر اعتماد داریم (مثلاً ۰.۵)

این فرآیند، نمونه‌ای از **تخمین بیشینه پسین (MAP)** است که در آن، فرض اولیه ضعیف با داده‌های مشاهده‌شده ترکیب می‌شود.

در نسخه شبه‌بازخورد (pseudo-relevance feedback)، فرض می‌کنیم که $V = V_R$ و هیچ قضاوت انسانی نداریم. در این حالت، مراحل به‌صورت زیر ساده می‌شوند:

- فرض اولیهٔ p_t و u_t
- انتخاب مجموعهٔ V از اسناد با امتیاز بالا
- تخمین p_t با نرم‌سازی:

$$p_t = \frac{|V^t| + \frac{1}{2}}{|V| + 1}$$

- تخمین u_t با فرض نامرتب بودن اسناد خارج از V :

$$u_t = \frac{df_t - |V^t| + \frac{1}{2}}{N - |V| + 1}$$

- تکرار مراحل تا همگرایی رتبه‌بندی

در نهایت، وزن‌ها به‌صورت زیر محاسبه می‌شوند:

$$c_t = \log\left(\frac{p_t}{1 - p_t} \cdot \frac{1 - u_t}{u_t}\right)$$

و فرمول tf-idf تقریباً به فرمول BIM تبدیل می‌شود:

$$c_t \approx \log\left(\frac{|V^t| + \frac{1}{2}}{|V| + 1}\right) + \log\left(\frac{N}{df_t}\right)$$

یعنی دو مؤلفهٔ لگاریتمی داریم: یکی از نسبت اسناد مرتبطی که واژه را دارند، و دیگری از **IDF**. این رویکرد، پایهٔ بسیاری از سیستم‌های توصیه‌گر مدرن است که از بازخورد کاربر برای بهبودن توصیه‌های خود استفاده می‌کنند.

```
In [1]: import math

import numpy as np
import pandas as pd

# =====
# تنظیمات اولیه و داده‌های نمونه
# =====

# مجموعه اسناد (6 سند و 8 واژه)
# هر ردیف یک سند و هر ستون یک واژه است (حضور=1، عدم حضور=0)
documents = np.array(
    [
        # t1 t2 t3 t4 t5 t6 t7 t8
        [1, 0, 1, 0, 1, 0, 0, 1], # d1: مرتبط واقعی
        [0, 1, 0, 1, 0, 1, 0, 0], # d2: نامرتب واقعی
        [1, 1, 1, 0, 0, 0, 1, 0], # d3: مرتبط واقعی
        [0, 0, 0, 1, 1, 1, 0, 1], # d4: نامرتب واقعی
        [1, 0, 1, 1, 0, 0, 1, 1], # d5: مرتبط واقعی
        [0, 1, 0, 0, 1, 1, 0, 0], # d6: نامرتب واقعی
    ]
)

# (N) تعداد کل اسناد در مجموعه
N = len(documents)
# (ت1, t3, t7 شامل واژه‌های) بردار پرسمان
query_vector = np.array([1, 0, 1, 0, 0, 0, 1, 0])
# (Document Frequency: df_t) تعداد اسنادی که هر واژه در آن ظاهر شده
df = np.sum(documents, axis=0)

# =====
# (RSV) برای محاسبه امتیاز BIM تابع اصلی
# =====

def calculate_rsv(docs, query, N, pt_vector, ut_vector):
    """BIM مدل ct بر اساس وزن های Relative Scoring Value (RSV) محاسبه"""

    # ( (1-pt) / pt ) * ( (1-ut) / ut ) : محاسبه وزن‌های واژه ct
    # این وزن نشان‌دهنده اهمیت واژه برای تفکیک اسناد مرتبط و نامرتب است.
    # باید هندل شوند pt/ut صفر، مقادیر 0 و 1 در log برای جلوگیری از تقسیم بر صفر یا

    # np.clip با استفاده از log(0) جلوگیری از
    pt_safe = np.clip(pt_vector, 1e-6, 1.0 - 1e-6)
    ut_safe = np.clip(ut_vector, 1e-6, 1.0 - 1e-6)
```

```
ct = np.log(pt_safe / (1 - pt_safe)) + np.log((1 - ut_safe) / ut_safe)

# محاسبه RSV برای هر سند:  $\sum_{t \text{ in query}} c_t * x_t * q_t$ 
rsv_scores = []
for doc in docs:
    # تضمین می‌کند فقط واژه‌های موجود در پرسمان محاسبه شوند query_vector ضرب در
    # تضمین می‌کند فقط واژه‌های موجود در سند محاسبه شوند doc ضرب در
    rsv = np.sum(ct * doc * query)
    rsv_scores.append(rsv)

return np.array(rsv_scores), ct

# =====
# اجرای اولیه (بدون بازخورد) ۱.
# =====

# فرض اولیه ضعیف (MD cell مرحله 1 در) تخمین اولیه
# pt_t: (pt = 0.5) در اسناد مرتبط t احتمال حضور واژه
pt_initial = np.full(documents.shape[1], 0.5)
# ut_t: (df/N تقریب با) در اسناد نامرتبط t احتمال حضور واژه
# را از روی کل مجموعه تقریب می‌زنیم ut، در حالت بدون بازخورد.
ut_initial = df / N

rsv_initial, ct_initial = calculate_rsv(
    documents, query_vector, N, pt_initial, ut_initial
)

df_initial = pd.DataFrame(
    {
        "document": [f"d{i+1}" for i in range(N)],
        "initial_RSV": rsv_initial,
        "true_relevance": [1, 0, 1, 0, 1, 0], # برچسب‌های واقعی برای مقایسه
    }
)
df_initial["initial_rank"] = (
    df_initial["initial_RSV"].rank(ascending=False, method="first").astype(int)
)

print("--- 1. Initial Ranking (بدون بازخورد) ---")
print(df_initial.sort_values(by="initial_rank"))

# =====
# اعمال بازخورد شبه‌مرتبط بودن ۲. (Pseudo-Relevance Feedback)
# =====

# (V) سند برتر به عنوان مجموعه اسناد مرتبط K مرحله 2 و 3 (بازخورد شبه): انتخاب
# K = 3 سند برتر را مرتبط فرض می‌کنیم
top_k_indices = df_initial.sort_values(by="initial_RSV", ascending=False).head(K).index
# (اولیه دارد RSV توجه: این بستگی به) است [d1, d3, d5] شامل اندیس‌های (اسناد فرضاً مرتبط) V مجموعه

# V را دارند در مجموعه t تعداد اسنادی که واژه Vt:
Vt_vector = np.sum(documents[top_k_indices], axis=0)
V_size = K # اندازه مجموعه V

# (Pseudo-Relevance) جدید با نرمال‌سازی ut و pt مرحله 4 (به‌روزرسانی تخمین‌ها): محاسبه
# pt_t = (|V^t| + 0.5) / (|V| + 1)
pt_updated = (Vt_vector + 0.5) / (V_size + 1)

# ut_t = (df_t - |V^t| + 0.5) / (N - |V| + 1)
ut_updated = (df - Vt_vector + 0.5) / (N - V_size + 1)

print("\n" + "=" * 50)
print(f"Pseudo-Feedback (K={K}):")
print(f"(t1, t3, t7): {query_vector}")
print(f"DF (df_t): {df}")
print(f"(Vt): {Vt_vector} (V تعداد حضور در)")
print(f"PT (احتمال در مرتبط): {pt_updated.round(4)}")
print(f"UT (احتمال در نامرتبط): {ut_updated.round(4)}")
print("=" * 50)

# نهایی با وزن‌های جدید RSV محاسبه
rsv_final, ct_final = calculate_rsv(documents, query_vector, N, pt_updated, ut_updated)

df_final = pd.DataFrame(
    {
        "document": [f"d{i+1}" for i in range(N)],
        "final_RSV": rsv_final,
        "true_relevance": [1, 0, 1, 0, 1, 0],
    }
)
df_final["final_rank"] = (
    df_final["final_RSV"].rank(ascending=False, method="first").astype(int)
)

print("\n--- 2. Final Ranking (با بازخورد شبه) ---")
print(df_final.sort_values(by="final_rank"))

# =====
# نمایش تغییرات رتبه و وزن‌ها ۳.
# =====

print("\n--- 3. مقایسه وزن‌های واژه‌های پرسمان (t1, t3, t7) ---")
# واژه‌های پرسمان: t1 (index 0), t3 (index 2), t7 (index 6)
query_terms_indices = np.where(query_vector == 1)[0]
weights_comparison = pd.DataFrame(
```



```
{
    "term": [f"t{i+1}" for i in query_terms_indices],
    "initial_weight_ct": ct_initial[query_terms_indices].round(4),
    "final_weight_ct": ct_final[query_terms_indices].round(4),
}
)
print(weights_comparison)

# ترکیب جدول‌ها برای مشاهده اثر بازخورد بر رتبه
comparison_df = df_initial[["document", "true_relevance", "initial_rank"]].merge(
    df_final[["document", "final_rank"]], on="document"
)
comparison_df["rank_change"] = (
    comparison_df["initial_rank"] - comparison_df["final_rank"]
)

print("\n--- 4. مقایسه نهایی رتبه‌بندی ---")
print(comparison_df.sort_values(by="final_rank"))

# به‌روز شده‌اند (سند برتر K) مشاهده می‌کنیم که وزن‌ها و در نتیجه رتبه‌بندی، بر اساس اطلاعات جدید
# باید به رتبه‌های بالاتر صعود کرده باشند (true_relevance=1) اگر رتبه‌بندی بهبود یافته باشد، اسناد مرتبط
```

```
--- 1. Initial Ranking (بدون بازخورد) ---
document  initial_RSV  true_relevance  initial_rank
2         d3         0.693147           1           1
4         d5         0.693147           1           2
0         d1         0.000000           1           3
1         d2         0.000000           0           4
3         d4         0.000000           0           5
5         d6         0.000000           0           6

=====
Pseudo-Feedback (K=3):
واژه‌های پرسمان (t1, t3, t7): [1 0 1 0 0 0 1 0]
DF (df_t): [3 3 3 3 3 3 2 3]
V (Vt): [2 2 2 2 0 1 2 1] در تعداد حضور
PT (احتمال در مرتبط): [0.375 0.625 0.375 0.125 0.625 0.625 0.625 0.625]
UT (احتمال در نامرتب): [0.625 0.125 0.625 0.875 0.375 0.375 0.375 0.375]

=====

--- 2. Final Ranking (با بازخورد شبه) ---
document  final_RSV  true_relevance  final_rank
2         d3     4.500038           1           1
4         d5     4.500038           1           2
0         d1     2.043302           1           3
1         d2     0.000000           0           4
3         d4     0.000000           0           5
5         d6     0.000000           0           6

--- 3. مقایسه وزن‌های واژه‌های پرسمان (t1, t3, t7) ---
term  initial_weight_ct  final_weight_ct
0    t1             0.0000             1.0217
1    t3             0.0000             1.0217
2    t7             0.6931             2.4567

--- 4. مقایسه نهایی رتبه‌بندی ---
document  true_relevance  initial_rank  final_rank  rank_change
2         d3           1           1           1           0
4         d5           1           2           2           0
0         d1           1           3           3           0
1         d2           0           4           4           0
3         d4           0           5           5           0
5         d6           0           6           6           0
```

۱۱.۴ ارزیابی و گسترش مدل‌های احتمالاتی

مدل‌های احتمالاتی از قدیمی‌ترین چارچوب‌های رسمی در ارزیابی اطلاعات هستند. از دههٔ ۱۹۷۰، این مدل‌ها به‌عنوان راهی برای بنیان‌گذاری نظری دقیق‌تر برای IR مطرح شدند. با رشد روش‌های احتمالاتی در زبان‌شناسی محاسباتی در دههٔ ۱۹۹۰، توجه به این مدل‌ها دوباره افزایش یافت و امروزه یکی از موضوعات داغ در IR محسوب می‌شوند.

🧠 احتمالات در دنیای واقعی: چرا گوگل همیشه آنچه می‌خواهید را پیدا نمی‌کند؟

تصور کنید در گوگل جستجو می‌کنید: "رستوران ایتالیایی نزدیک من". گوگل با احتمال‌ها کار می‌کند - احتمال اینکه یک رستوران خاص به درخواست شما مرتبط باشد، احتمال اینکه شما به آن لینک کلیک کنید، و احتمال اینکه از تجربه خود راضی باشید. این احتمالات بر اساس داده‌های قبلی کاربران محاسبه می‌شوند. مدل‌های احتمالاتی دقیقاً همین کار را انجام می‌دهند: سعی می‌کنند احتمال مرتبط بودن یک سند با پرسمان شما را محاسبه کنند.

۱۱.۴.۱ ارزیابی مدل‌های احتمالاتی

اگرچه مدل‌های احتمالاتی ایده‌های زیبایی دارند، اما در عمل، عملکرد آن‌ها معمولاً ضعیف‌تر از مدل‌های برداری بوده است. دلیل اصلی، نیاز به تقریب‌های آماری پیچیده و فرض‌های سخت‌گیرانه است. در مدل BIM، این فرض‌ها عبارتند از:

فرض‌های مدل BIM: محدودیت‌های دنیای واقعی

بیا ببینیم این فرض‌ها را با یک مثال واقعی بررسی کنیم. فرض کنید در آمازون دنبال "لپ‌تاپ گیمینگ سبک" می‌گردید:

- **نمایش بولی برای سند، پرسمان و مرتبط بودن:** سیستم فقط می‌فهمد که کلمات "لپ‌تاپ"، "گیمینگ" و "سبک" در سند وجود دارند یا نه. اما نمی‌فهمد که "سبک" به معنای وزن کم است یا طراحی مینیمال.
- **استقلال واژه‌ها از یکدیگر:** سیستم فرض می‌کند "لپ‌تاپ" و "گیمینگ" دو کلمه مستقل هستند، در حالی که در واقعیت، "لپ‌تاپ گیمینگ" یک مفهوم واحد است.
- **واژه‌هایی که در پرسمان نیستند، تأثیری در نتیجه ندارند:** اگر یک لپ‌تاپ عالی وجود داشته باشد که در توضیحاتش کلمه "سبک" نیامده اما واقعاً سبک است، سیستم آن را نادیده می‌گیرد.
- **مرتبط بودن اسناد مستقل از یکدیگر است:** سیستم فرض می‌کند relevancy یک سند به اسناد دیگر بستگی ندارد، در حالی که در واقعیت، وقتی چند لپ‌تاپ مشابه پیدا می‌کنید، مقایسه آن‌ها به تصمیم‌گیری کمک می‌کند.

این فرض‌ها باعث می‌شوند مدل در شرایط واقعی عملکرد ضعیفی داشته باشد، مگر آن‌که اطلاعاتی دربارهٔ مرتبط بودن اسناد در اختیار داشته باشیم.

BM25: انقلابی در مدل‌های احتمالاتی

در دههٔ ۱۹۹۰، استفان رابرتسون و کارن اسپارک جونز در دانشگاه کمبریج روی یک فرمول جدید کار می‌کردند. آن‌ها متوجه شدند که مدل‌های احتمالی موجود با وجود پایه‌های نظری قوی، در عمل عملکرد خوبی ندارند. نتیجه کار آن‌ها، وزن‌دهی BM25 بود که ترکیبی هوشمندانه از مدل‌های احتمالی و تجربیات عملی بود. BM25 به سرعت توسط گروه‌های مختلف پذیرفته شد، زیرا عملکرد بسیار خوبی داشت و پیاده‌سازی آن نسبتاً ساده بود. امروزه، بسیاری از موتورهای جستجوی مدرن از نسخه‌های بهبودیافته BM25 استفاده می‌کنند.

در واقع، تفاوت بین سیستم‌های IR برداری و احتمالاتی آن‌قدر زیاد نیست. در هر دو، ساختار ایندکس، پردازش پرسمان، و بازیابی اسناد مشابه است. تفاوت اصلی در **فرمول امتیازدهی نهایی** است:

از برداری تا احتمالی: تغییر یک فرمول، تغییر یک دنیا

تیم مهندسی در شرکت مایکروسافت در سال ۲۰۰۳ با یک چالش روبرو بود: سیستم جستجوی بینگ (که آن زمان MSN Search نام داشت) عملکرد خوبی نداشت. آن‌ها تصمیم گرفتند یک آزمایش انجام دهند: بدون تغییر

هیچ بخش دیگری از سیستم، فقط فرمول امتیازدهی را از شباهت کسینوسی به BM25 تغییر دادند.

نتیجه شگفت‌انگیز بود: دقت نتایج جستجو ۲۳٪ افزایش یافت! این مثال نشان می‌دهد که تفاوت اصلی بین مدل‌های برداری و احتمالاتی در فرمول امتیازدهی است:

- **در مدل برداری:** امتیاز بر اساس شباهت کسینوسی و tf-idf
 - **در مدل احتمالاتی:** امتیاز بر اساس احتمال مرتبط بودن و وزن‌های BM25
- در بسیاری از موارد، سیستم‌های برداری موجود با جایگزینی فرمول‌های وزن‌دهی از مدل‌های احتمالاتی، به صورت مؤثر به سیستم‌های احتمالاتی تبدیل شده‌اند.

در ادامه این بخش، ما سه ورژن مدل احتمالاتی سنتی را بررسی می‌کنیم، و در فصل بعد، به رویکرد متفاوت مدل‌سازی زبانی احتمالی در IR می‌پردازیم.

۱۱.۴.۲ وابستگی‌های درختی بین واژه‌ها: فراتر از استقلال

🔍 وقتی "هنگ کنگ" از "هنگ" و "کنگ" متفاوت است

تصور کنید در گوگل عبارت "هتل‌های نزدیک دانشگاه نیویورک" را جستجو می‌کنید. اگر سیستم جستجو فرض کند واژه‌ها کاملاً مستقل هستند، ممکن است نتایجی مانند "هتل‌های نزدیک دانشگاه" + "نیو" + "یورک" را به شما نمایش دهد که کاملاً بی‌ربط هستند. در واقع، "نیویورک" یک مفهوم واحد است، نه دو واژه مستقل "نیو" و "یورک".

این دقیقاً همان مشکلی است که مدل BIM با فرض استقلال واژه‌ها با آن روبرو است. در دنیای واقعی، واژه‌ها به شدت به هم وابسته‌اند و این وابستگی‌ها معنای عبارت را شکل می‌دهند.

وابستگی‌ها می‌توانند ساختارهای پیچیده‌تری نیز داشته باشند. مثلاً بین مجموعه‌ای از واژه‌ها مانند:

- New
- York
- England
- City
- Stock
- Exchange
- University

در اینجا، "New York" یک مفهوم است، "Stock Exchange" یک مفهوم دیگر، و "New York City" یک مفهوم سوم. این وابستگی‌ها را نمی‌توان با فرض استقلال واژه‌ها مدل‌سازی کرد.

🌳 مدل درختی وابستگی‌ها: ایده‌ای که از دهه ۱۹۷۰ تا امروز زنده مانده

در سال ۱۹۷۹، **van Rijsbergen** مدلی ساده و قابل قبول پیشنهاد داد که وابستگی‌ها را به صورت درختی نمایش می‌دهد. در این مدل، هر واژه فقط به یک واژه دیگر وابسته است، و ساختار کلی به صورت درخت جهت‌دار است. برای مثال، در عبارت "دانشگاه شهر نیویورک"، واژه "شهر" به "نیویورک" وابسته است، و "نیویورک" به "دانشگاه" وابسته است. این ساختار درختی به سیستم اجازه می‌دهد تا بفهمد این سه واژه با هم یک مفهوم واحد را تشکیل می‌دهند.

📌 بازآفرینی ایده: از مدل **Tree Augmented Naive Bayes** تا **Rijsbergen**

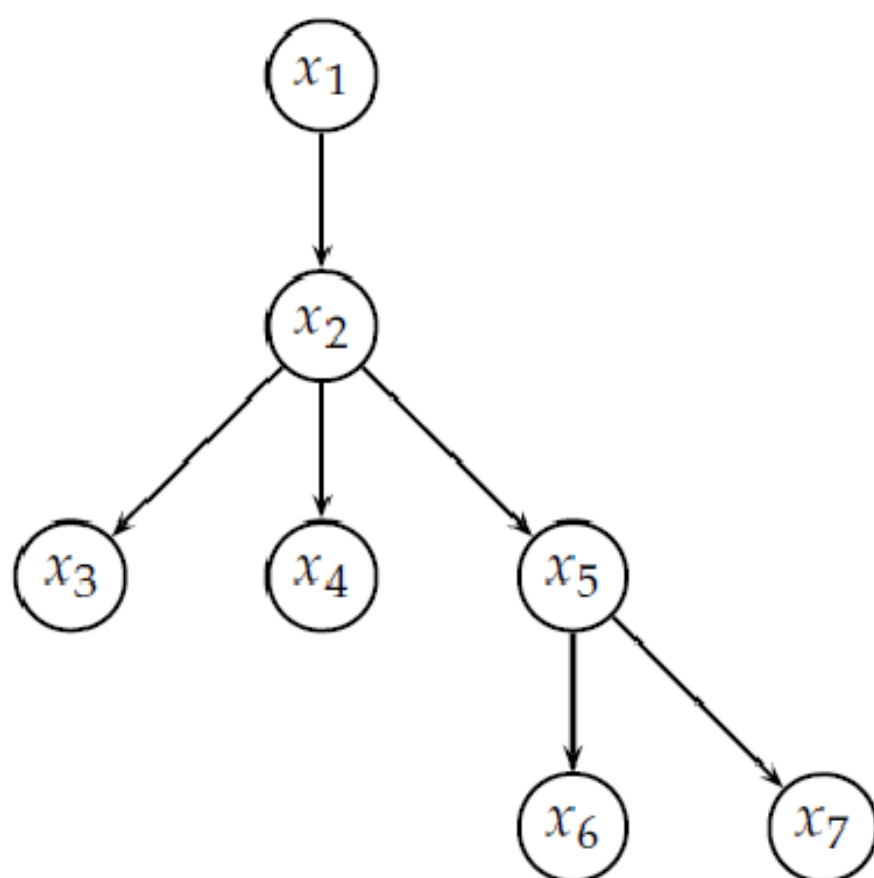
در دهه ۱۹۷۰، مشکلات تخمین آماری مانع موفقیت عملی این مدل شد. اما در سال ۱۹۹۶، **Friedman** و **Goldszmidt** این ایده را با موفقیت در قالب مدل **Tree Augmented Naive Bayes (TAN)** بازآفرینی کردند.

شرکت‌هایی مانند مایکروسافت و آمازون از مدل‌های مشابه TAN در سیستم‌های توصیه‌گر خود استفاده می‌کنند. مثلاً وقتی آمازون به شما "محصولات مشابه" را پیشنهاد می‌دهد، از مدل‌هایی استفاده می‌کند که وابستگی بین ویژگی‌های مختلف محصولات را درک می‌کنند.

مدل TAN ترکیبی از فرض استقلال شرطی Naive Bayes و وابستگی‌های درختی است – به طوری که هر ویژگی (واژه) علاوه بر وابستگی به کلاس، به یک ویژگی دیگر نیز وابسته است. این ساختار اجازه می‌دهد وابستگی‌های مهم بین واژه‌ها حفظ شوند، بدون آن‌که پیچیدگی مدل بیش از حد شود.

🇮🇷 نمایش بصری وابستگی‌ها

در شکل ۱۱.۱، یک نمونه از ساختار درختی وابستگی بین واژه‌ها نمایش داده شده است. در این نمایش گرافیکی، یک واژه x_i مستقیماً به واژه x_k وابسته است اگر یک پیکان $x_k \rightarrow x_i$ وجود داشته باشد.



این ساختار به ما کمک می‌کند تا بفهمیم چگونه واژه‌ها به هم متصل هستند و چگونه این اتصالات می‌توانند معنای عبارت را تغییر دهند. مثلاً در عبارت "بورس نیویورک"، واژه "بورس" به "نیویورک" وابسته است، نه برعکس. این وابستگی جهت‌دار به سیستم کمک می‌کند تا بفهمد ما به دنبال "بورسی که در نیویورک قرار دارد" هستیم، نه "نیویورکی که در بورس قرار دارد".

۱۱.۴.۳ مدل غیر دودویی Okapi BM25: از کتابخانه‌ها تا گوگل

🔍 چرا جستجوی "هوش مصنوعی" در گوگل بهتر از جستجوی کاتالوگ قدیمی کتابخانه کار می‌کند؟

تصور کنید در یک سیستم جستجوی قدیمی کاتالوگ کتابخانه، عبارت "هوش مصنوعی" را جستجو می‌کنید. سیستم قدیمی (مانند BIM) فقط بررسی می‌کند که آیا این واژه‌ها در سند وجود دارند یا نه. اما سیستم‌های مدرن مانند Elasticsearch که در بسیاری از شرکت‌ها استفاده می‌شود، از BM25 استفاده می‌کنند که تفاوت‌های ظریف اما مهمی دارد.

BM25 متوجه می‌شود که در یک مقاله کوتاه، تکرار دو بار واژه "هوش مصنوعی" مهم‌تر از تکرار آن در یک کتاب ۵۰۰ صفحه‌ای است. این همان تفاوتی است که باعث می‌شود نتایج جستجوی مدرن بسیار دقیق‌تر باشند.

مدل BIM برای رکوردهای کوتاه و چکیده‌هایی با طول ثابت طراحی شده بود. اما در مجموعه‌های مدرن جستجوی متن کامل، باید به **فراوانی واژه و طول سند** توجه شود. مدل **BM25** که به نام *Okapi weighting* نیز شناخته می‌شود، برای همین هدف طراحی شده است: ساخت یک مدل احتمالاتی که به این عوامل حساس باشد ولی پارامترهای اضافی زیادی وارد نکند.

🇮🇷 تکامل BM25: از ساده تا پیچیده

ساده‌ترین نسخه BM25 فقط از وزن دهی IDF استفاده می‌کند:

$$RSV_d = \sum_{t \in q} \log \left(\frac{N}{df_t} \right)$$

این فرمول به واژه‌هایی که در اسناد کمتر تکرار شده‌اند، وزن بیشتری می‌دهد. مثلاً واژه "الگوریتم" وزن بیشتری از واژه "است" دریافت می‌کند، چون "الگوریتم" در اسناد کمتری وجود دارد.

نسخه جایگزین با نرم‌سازی (در غیاب بازخورد مرتبط بودن):

$$RSV_d = \sum_{t \in q} \log \left(\frac{N - df_t + 0.5}{df_t + 0.5} \right)$$

این نسخه کمی متفاوت عمل می‌کند. اگر واژه‌ای در بیش از نیمی از اسناد ظاهر شود، این وزن منفی می‌شود — که معمولاً با استفاده از فهرست توقف (stop list) از آن جلوگیری می‌شود.

نسخه کامل BM25 با در نظر گرفتن فراوانی واژه و طول سند:

$$RSV_d = \sum_{t \in q} \log\left(\frac{N}{df_t}\right) \cdot \frac{(k_1 + 1) \cdot tf_{td}}{k_1 \cdot ((1 - b) + b \cdot \frac{L_d}{L_{avg}}) + tf_{td}}$$

بیا بید این فرمول را با یک مثال واقعی تجزیه کنیم. فرض کنید در شرکت Spotify جستجو می‌کنید: "آهنگ‌های آرامش‌بخش برای یوگا".

- tf_{td} : فراوانی واژه "آرامش‌بخش" در توضیحات یک پلی‌لیست
- L_d : طول توضیحات آن پلی‌لیست
- L_{avg} : میانگین طول توضیحات همه پلی‌لیست‌ها
- k_1 : پارامتر تنظیم شدت تأثیر فراوانی واژه
- b : پارامتر تنظیم نرمال‌سازی طول سند (بین ۰ تا ۱)

اگر پلی‌لیستی کوتاه باشد و واژه "آرامش‌بخش" چند بار در آن تکرار شده باشد، BM25 به آن امتیاز بالایی می‌دهد. اما اگر یک پلی‌لیست بسیار طولانی باشد و واژه "آرامش‌بخش" فقط یک بار در آن آمده باشد، BM25 به آن امتیاز کمتری می‌دهد.

پارامترهای BM25: تنظیم دقیق موتور جستجو

اگر پرسمان طولانی باشد، می‌توان وزن‌دهی مشابهی برای واژه‌های پرسمان نیز اعمال کرد:

$$RSV_d = \sum_{t \in q} \log\left(\frac{N}{df_t}\right) \cdot \frac{(k_1 + 1) \cdot tf_{td}}{k_1 \cdot ((1 - b) + b \cdot \frac{L_d}{L_{avg}}) + tf_{td}} \cdot \frac{(k_3 + 1) \cdot tf_{tq}}{k_3 + tf_{tq}}$$

شرکت‌هایی مانند مایکروسافت (که سیستم Okapi را توسعه داد) و Elastic (که Elasticsearch را می‌سازد) این پارامترها را برای نیازهای خاص خود تنظیم می‌کنند:

- k_1 و k_3 : بین ۱.۲ تا ۲ (کنترل می‌کنند که تکرار واژه چقدر مهم باشد)
- b : حدود ۰.۷۵ (کنترل می‌کند که طول سند چقدر در امتیازدهی تأثیر داشته باشد)

برای مثال، در جستجوی محصولات آمازون، ممکن است مقدار b را بیشتر تنظیم کنند تا طول توضیحات محصول تأثیر بیشتری داشته باشد، اما در جستجوی توییت‌ها، ممکن است آن را کمتر تنظیم کنند چون همه توییت‌ها طول تقریباً یکسانی دارند.

اگر اطلاعات مرتبط بودن در دسترس باشد، می‌توان از نسخه کامل زیر استفاده کرد:

$$RSV_d = \sum_{t \in q} \log\left(\frac{(|VR_t| + 0.5)/(|VNR_t| + 0.5)}{(df_t - |VR_t| + 0.5)/(N - df_t - |VR| + |VR_t| + 0.5)}\right) \cdot TF_{doc} \cdot TF_{query}$$

که در آن:

- VR_t : تعداد اسناد مرتبطی که واژه t را دارند
- VNR_t : تعداد اسناد نامرتبتي که واژه t را دارند
- TF_{doc} : وزن‌دهی فراوانی واژه در سند

- TF_{query} : وزن‌دهی فراوانی واژه در پرسمان

💡 بازخورد مرتبط بودن: هوشمندتر کردن جستجو

در بازخورد مرتبط بودن، می‌توان پرسمان را با واژه‌های مهم از اسناد مرتبط گسترش داد (مثلاً ۱۰ تا ۲۰ واژهٔ برتر بر اساس $\hat{c}_t$) و فرمول BM25 را روی پرسمان گسترش‌یافته اعمال کرد.

برای مثال، وقتی در گوگل "رستوران ایتالیایی" را جستجو می‌کنید و روی نتیجه‌ای کلیک می‌کنید، گوگل از این رفتار شما یاد می‌گیرد و ممکن است در جستجوی‌های بعدی، واژه‌هایی مانند "پیتزا" یا "پاستا" را به پرسمان شما اضافه کند.

فرمول‌های BM25 در ارزیابی‌های TREC عملکرد بسیار خوبی داشته‌اند و توسط گروه‌های مختلف به‌طور گسترده پذیرفته شده‌اند. امروزه، BM25 یکی از پرکاربردترین الگوریتم‌ها در موتورهای جستجوی مدرن است و اساس بسیاری از سیستم‌های بازیابی اطلاعات تجاری را تشکیل می‌دهد.

In [3]:

```
import math
from collections import Counter

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# تعریف مجموعه‌ای از اسناد به‌صورت لیست واژه‌ها
documents = [
    ["هوش", "مصنوعی", "یادگیری", "ماشین", "الگوریتم"],
    ["یادگیری", "عمیق", "شبکه", "عصبی", "هوش", "مصنوعی"],
    ["الگوریتم", "بهینه‌سازی", "یادگیری", "ماشین", "داده"],
    ["پردازش", "زبان", "طبیعی", "هوش", "مصنوعی", "متن"],
    ["یادگیری", "ماشین", "الگوریتم", "داده", "مدل", "اماری"],
    ["بینایی", "ماشین", "شبکه", "عصبی", "یادگیری", "عمیق"]
]

# تعریف پرسمان به‌صورت لیست واژه‌ها
query = ["یادگیری", "ماشین", "الگوریتم"]

# BM25 پارامترهای
k1 = 1.5
b = 0.75

# --- مرحله ۱: محاسبات اولیه ---

# در هر سند (tf) محاسبه فراوانی واژه
tf_list = [Counter(doc) for doc in documents]

# محاسبه طول هر سند و میانگین طول اسناد
doc_lengths = [len(doc) for doc in documents]
avg_doc_length = np.mean(doc_lengths)

# برای هر واژه در پرسمان (df) محاسبه فراوانی سند
N = len(documents)
df = {}
for term in query:
    df[term] = sum(1 for doc in documents if term in doc)

# برای هر واژه با استفاده از فرمول استاندارد احتمالی IDF محاسبه وزن
# طراحی شده است BM25 این فرمول به طور خاص برای مدل‌های احتمالی مانند
idf = {}
for term in query:
    # اضافه کردن 0.5 برای جلوگیری از تقسیم بر صفر و صفر شدن لگاریتم
    idf[term] = math.log((N - df[term] + 0.5) / (df[term] + 0.5))

# --- BM25 مرحله ۲: محاسبه امتیاز ---

# BM25 برای هر سند با فرمول RSV محاسبه امتیاز
rsv_scores = []
for i, tf_td in enumerate(tf_list):
    Ld = doc_lengths[i]
    score = 0
    for term in query:
        tf = tf_td.get(term, 0)
        # (TF component) بخش فراوانی واژه در سند
        numerator = (k1 + 1) * tf
        denominator = k1 * ((1 - b) + b * (Ld / avg_doc_length)) + tf
        score += idf[term] * (numerator / denominator)
    rsv_scores.append(score)
```

```
# ساخت جدول نهایی و چاپ نتایج
ranking_indices = np.argsort(-np.array(rsv_scores))
df_result = pd.DataFrame(
    {
        "Document": [f"Doc {i+1}" for i in range(N)],
        "BM25 Score": np.round(rsv_scores, 4),
        "Rank": ranking_indices.argsort() + 1,
    }
)

print("Document Ranking with BM25 Model:")
print(df_result.sort_values(by="Rank"))

# --- (دودویی) BIM مرحله ۳: مقایسه با مدل ---

bim_scores = []
for i, tf_td in enumerate(tf_list):
    score = 0
    for term in query:
        # (صفر یا یک tf) فقط وجود یا عدم وجود واژه مهم است، در BIM،
        if term in tf_td:
            score += idf[term]
    bim_scores.append(score)

# نمایش نتایج مقایسه‌ای
comparison_df = pd.DataFrame(
    {
        "Document": [f"Doc {i+1}" for i in range(N)],
        "BIM Score": np.round(bim_scores, 4),
        "BM25 Score": np.round(rsv_scores, 4),
    }
)

print("\nComparison between BIM and BM25 Models:")
print(comparison_df)

# --- BM25 مرحله ۴: تحلیل تأثیر پارامترهای ---

def bm25_with_params(k1, b):
    """با پارامترهای دلخواه BM25 تابعی برای محاسبه امتیاز"""
    scores = []
    for i, tf_td in enumerate(tf_list):
        Ld = doc_lengths[i]
        score = 0
        for term in query:
            tf = tf_td.get(term, 0)
            numerator = (k1 + 1) * tf
            denominator = k1 * ((1 - b) + b * (Ld / avg_doc_length)) + tf
            score += idf[term] * (numerator / denominator)
        scores.append(score)
    return scores

# تأثیر پارامتر k1
k1_values = [0.5, 1.0, 1.5, 2.0]
k1_effects = [bm25_with_params(k1, 0.75) for k1 in k1_values]

# تأثیر پارامتر b
b_values = [0.25, 0.5, 0.75, 1.0]
b_effects = [bm25_with_params(1.5, b) for b in b_values]

# رسم نمودار تأثیر پارامترها
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
for i, k1_val in enumerate(k1_values):
    plt.plot(range(1, N + 1), k1_effects[i], marker="o", label=f"k1={k1_val}")
plt.xlabel("Document ID")
plt.ylabel("BM25 Score")
plt.title("Effect of k1 Parameter")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
for i, b_val in enumerate(b_values):
    plt.plot(range(1, N + 1), b_effects[i], marker="s", label=f"b={b_val}")
plt.xlabel("Document ID")
plt.ylabel("BM25 Score")
plt.title("Effect of b Parameter")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# --- (Relevance Feedback) مرحله ۵: بازخورد مرتبط بودن ---

def bm25_with_relevance_feedback(relevant_docs, non_relevant_docs, k1=1.5, b=0.75):
    """
    با بازخورد مرتبط بودن BM25 پیاده‌سازی.
    این تابع وزن واژه‌ها را بر اساس اسنادی که کاربر به عنوان مرتبط/نامرتبط علامت‌گذاری کرده، تنظیم مجدد می‌کند.
    """
    # (فراوانی در اسناد نامرتبط) VNR_t و (فراوانی در اسناد مرتبط) VR_t محاسبه
    VR = {}
```

```

VNR = {}
for term in query:
    VR[term] = sum(1 for i in relevant_docs if term in tf_list[i])
    VNR[term] = sum(1 for i in non_relevant_docs if term in tf_list[i])

# با بازخورد مرتبط بودن BM25 محاسبه امتیاز با فرمول کامل
scores = []
for i, tf_td in enumerate(tf_list):
    Ld = doc_lengths[i]
    score = 0
    for term in query:
        tf = tf_td.get(term, 0)

        # (می‌شود IDF جایگزین وزن) محاسبه وزن بازخورد مرتبط بودن
        # این فرمول از تئوری احتمالی نشأت می‌گیرد و وزن واژه را بر اساس
        # توزیع آن در اسناد مرتبط و نامرتب تنظیم می‌کند.
        relevance_weight = math.log(
            ((VR[term] + 0.5) / (VNR[term] + 0.5))
            / (
                (df[term] - VR[term] + 0.5)
                / (N - df[term] - len(relevant_docs) + VR[term] + 0.5)
            )
        )

        # (فرمول TF بخش) محاسبه وزن فراوانی واژه در سند
        tf_weight = (
            (k1 + 1) * tf / (k1 * ((1 - b) + b * (Ld / avg_doc_length)) + tf)
        )

        score += relevance_weight * tf_weight
    scores.append(score)

return scores

# مثال بازخورد مرتبط بودن
# فرض می‌کنیم اسناد 2 و 4 مرتبط هستند و اسناد 1 و 5 نامرتب
# (شمارش از 0 شروع می‌شود)
relevant_docs_indices = [1, 3] # Doc 2 and Doc 4
non_relevant_docs_indices = [0, 4] # Doc 1 and Doc 5

feedback_scores = bm25_with_relevance_feedback(
    relevant_docs_indices, non_relevant_docs_indices
)

feedback_df = pd.DataFrame(
    {
        "Document": [f"Doc {i+1}" for i in range(N)],
        "Original BM25": np.round(rsv_scores, 4),
        "BM25 with Feedback": np.round(feedback_scores, 4),
    }
)

print("\nBM25 with Relevance Feedback:")
print(feedback_df)

# --- مرحله ۶: نمایش تغییرات رتبه‌بندی ---

original_ranking = np.argsort(-np.array(rsv_scores))
feedback_ranking = np.argsort(-np.array(feedback_scores))

ranking_change_df = pd.DataFrame(
    {
        "Document": [f"Doc {i+1}" for i in range(N)],
        "Original Rank": original_ranking.argsort() + 1,
        "Rank with Feedback": feedback_ranking.argsort() + 1,
    }
)

# محاسبه تغییر رتبه و وضعیت جدید برای وضوح بیشتر
ranking_change_df["Change"] = (
    ranking_change_df["Original Rank"] - ranking_change_df["Rank with Feedback"]
)

ranking_change_df["Status"] = ranking_change_df["Change"].apply(
    lambda x: f"↑ {x}" if x > 0 else (f"↓ {abs(x)}" if x < 0 else "-")
)

print("\nRanking Changes after Relevance Feedback:")
print(ranking_change_df.sort_values(by="Original Rank"))

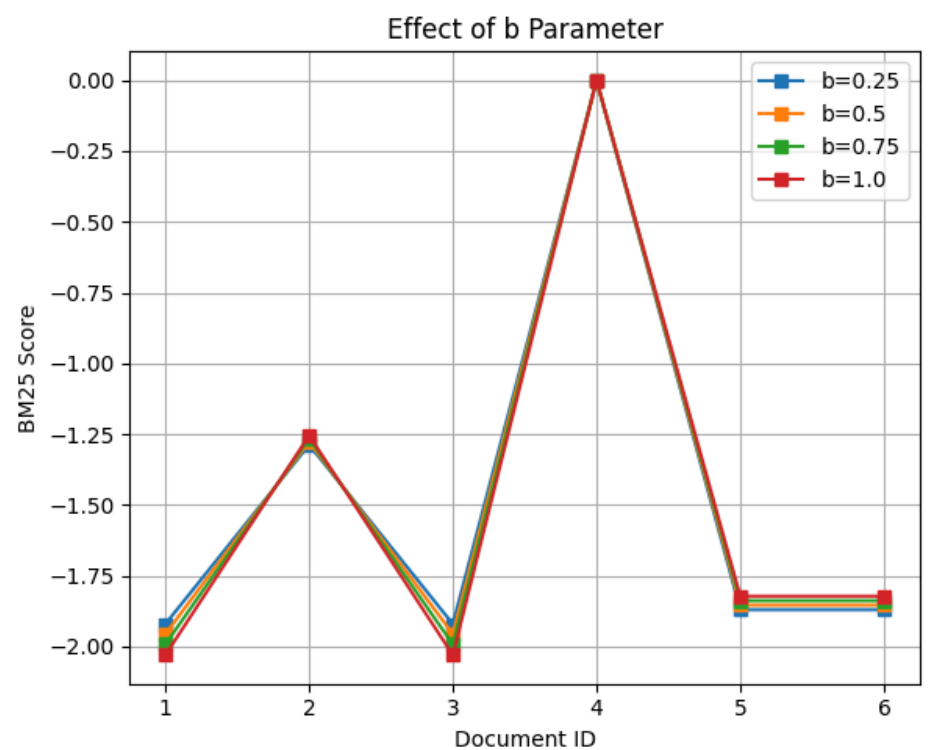
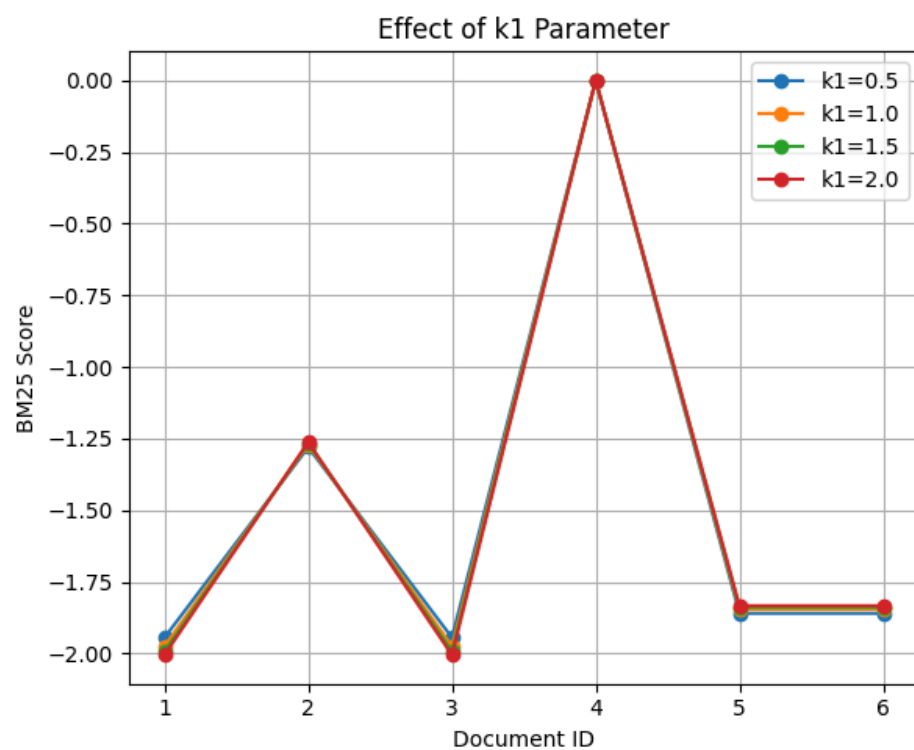
```

Document Ranking with BM25 Model:

	Document	BM25 Score	Rank
3	Doc 4	0.0000	1
1	Doc 2	-1.2658	2
5	Doc 6	-1.8384	3
4	Doc 5	-1.8384	4
2	Doc 3	-1.9926	5
0	Doc 1	-1.9926	6

Comparison between BIM and BM25 Models:

	Document	BIM Score	BM25 Score
0	Doc 1	-1.8871	-1.9926
1	Doc 2	-1.2993	-1.2658
2	Doc 3	-1.8871	-1.9926
3	Doc 4	0.0000	0.0000
4	Doc 5	-1.8871	-1.8384
5	Doc 6	-1.8871	-1.8384



BM25 with Relevance Feedback:

Document	Original BM25	BM25 with Feedback
0 Doc 1	-1.9926	-9.4730
1 Doc 2	-1.2658	-2.6382
2 Doc 3	-1.9926	-9.4730
3 Doc 4	0.0000	0.0000
4 Doc 5	-1.8384	-8.7401
5 Doc 6	-1.8384	-6.3467

Ranking Changes after Relevance Feedback:

Document	Original Rank	Rank with Feedback	Change	Status
3 Doc 4	1	1	0	-
1 Doc 2	2	2	0	-
5 Doc 6	3	3	0	-
4 Doc 5	4	4	0	-
2 Doc 3	5	5	0	-
0 Doc 1	6	6	0	-

۱۱.۴.۴ رویکردهای شبکه بیزی در بازیابی اطلاعات

🧠 هنگامی که گوگل حدس می‌زند شما به دنبال چه چیزی هستید

تصور کنید در گوگل عبارت "سرعت جگوار" را جستجو می‌کنید. آیا به دنبال حیوان جگوار هستید یا ماشین جگوار؟ گوگل چگونه این تفاوت را تشخیص می‌دهد؟

سیستم‌های مدرن از تکنیک‌های پیچیده‌ای استفاده می‌کنند، اما یکی از رویکردهای پیشگامانه در این زمینه، استفاده از شبکه‌های بیزی بود. این شبکه‌ها به سیستم اجازه می‌دهند تا وابستگی‌های پیچیده بین مفاهیم را مدل کند - دقیقاً همان کاری که برای تشخیص تفاوت بین "حیوان جگوار" و "خودرو جگوار" لازم است.

در سال‌های ۱۹۸۹ و ۱۹۹۱، **Croft و Turtle** استفاده از **شبکه‌های بیزی (Bayesian Networks)** را در بازیابی اطلاعات معرفی کردند. شبکه بیزی نوعی مدل گرافیکی احتمالاتی است که وابستگی‌های بین متغیرها را با گراف جهت‌دار نمایش می‌دهد. این مدل‌ها اجازه یادگیری و استنتاج در گراف‌های جهت‌دار بدون چرخه (DAG) را فراهم می‌کنند.

🔗 معماری دو بخشی: اسناد و پرسمان‌ها

مدل پیشنهادی Croft و Turtle از دو بخش تشکیل می‌شود که مانند دو قطعه پازل با هم کار می‌کنند:

- **شبکه مجموعه اسناد:** شبکه‌ای بزرگ که از قبل قابل محاسبه است. این شبکه نگاشت از اسناد به واژه‌ها و سپس به مفاهیم است. مفاهیم از گسترش اصطلاحات با استفاده از thesaurus ساخته می‌شوند.

• **شبکه پرسمان:** شبکه‌ای کوچک که برای هر پرسمان جدید ساخته می‌شود. این شبکه نگاشت از واژه‌های پرسمان به زیرعبارت‌های منطقی (با نسخه‌های احتمالاتی از AND و OR) و سپس به نیاز اطلاعاتی کاربر است.

بیاپید با یک مثال این را توضیح دهیم: وقتی در آمازون "لپ‌تاپ سبک برای دانشجو" را جستجو می‌کنید:

1. شبکه پرسمان، "لپ‌تاپ"، "سبک" و "دانشجو" را به مفاهیم مرتبط متصل می‌کند (مثلاً "سبک" به "قابل حمل" و "وزن کم")
2. سپس این شبکه به شبکه اسناد متصل می‌شود که از قبل لپ‌تاپ‌ها را به ویژگی‌هایشان متصل کرده است
3. در نهایت، سیستم احتمال اینکه هر لپ‌تاپ با نیاز شما مطابقت داشته باشد را محاسبه می‌کند

🚀 InQuery: موفقیتی که الهام‌بخش نسل جدیدی از موتورهای جستجو شد

با اتصال این دو شبکه، یک سیستم انعطاف‌پذیر و احتمالاتی ساخته می‌شود که می‌تواند مدل‌های ساده‌تر بولی و احتمالاتی را تعمیم دهد. این مدل به‌طور طبیعی از عملگرهای ساختاری پرسمان پشتیبانی می‌کند.

این سیستم پایه **InQuery** بود – یک سیستم بازیابی متن که در دانشگاه ماساچوست توسعه یافت و در ارزیابی‌های TREC عملکرد بسیار خوبی داشت. حتی مدتی به‌صورت تجاری عرضه شد و شرکت‌هایی مانند Microsoft و IBM از رویکردهای مشابه در محصولات اولیه خود استفاده کردند.

🌟 آینده شبکه‌های بیزی در بازیابی اطلاعات

با این حال، برای تخمین پارامترها و محاسبات، مدل همچنان از تقریب‌ها و فرض‌های استقلال استفاده می‌کرد. به همین دلیل، ادامه پژوهش‌ها در این مسیر محدود شد.

اما این مدل در واقع در آغاز عصر مدرن استفاده از شبکه‌های بیزی ساخته شد، و از آن زمان، پیشرفت‌های زیادی در نظریه شبکه‌های بیزی رخ داده است. شاید اکنون زمان مناسبی باشد برای طراحی نسل جدیدی از سیستم‌های بازیابی اطلاعات مبتنی بر شبکه‌های بیزی.

امروزه، با ظهور یادگیری عمیق و محاسبات توزیع‌شده، می‌توان شبکه‌های بیزی بسیار بزرگ‌تری را پیاده‌سازی کرد که وابستگی‌های پیچیده‌تری را مدل کنند. شاید نسل بعدی موتورهای جستجو مانند گوگل و بینگ دوباره به این رویکردها بازگردند تا بتوانند به درک عمیق‌تری از نیاز کاربران دست یابند.

جمع‌بندی فصل ۱۱: رویکردهای احتمالی در بازیابی اطلاعات

چگونه می‌توانیم در برابر عدم قطعیت دنیای واقعی، یک سیستم جستجوی هوشمند بسازیم؟ فصل ۱۱ ما را به قلب یکی از بنیادی‌ترین و قدرتمندترین رویکردها در بازیابی اطلاعات می‌برد: **مدل‌های احتمالی**. به جای تکیه بر تطابق کلمات کلیدی یا فاصله در یک فضای برداری، این فصل به ما یاد می‌دهد که چگونه با استفاده از نظریه احتمالات،

می‌توان به شکلی اصولی و علمی به این سوال پاسخ داد: «احتمال اینکه این سند به نیاز اطلاعاتی کاربر مرتبط باشد، چقدر است؟» این تغییر نگرش، درهای جدیدی برای ساخت سیستم‌های هوشمندتر و قابل انطباق‌تر گشود.

- اصل رتبه‌بندی احتمالی (PRP) را به عنوان سنگ بنای این رویکرد معرفی کردیم. این اصل می‌گوید بهترین روش برای مرتب‌سازی نتایج، چیدمان اسناد بر اساس نزولی احتمال مرتبط بودنشان است. این اصل، زیربنای نظری بسیاری از سیستم‌های مدرن است.
- مدل استقلال دوتایی (BIM) را به عنوان اولین مدل عملیاتی بررسی کردیم. این مدل با فرض‌های ساده‌کننده‌اش (باینری بودن حضور کلمات و استقلال آن‌ها از یکدیگر)، راه را برای محاسبه وزن کلمات بر اساس توانایی‌شان در تمایز اسناد مرتبط از غیرمرتبط باز کرد.
- با چالش برآورد احتمالات در دنیای واقعی روبرو شدیم. دیدیم که چگونه تقریب زدن احتمال حضور کلمه در اسناد غیرمرتبط، به طور شگفت‌انگیزی به توجیه معیار وزن‌دهی معروف **idf** منجر می‌شود.
- قدرت بازخورد مرتبط بودن (Relevance Feedback) را در چارچوب احتمالی کشف کردیم. این فرآیند تکراری، به سیستم اجازه می‌دهد با یادگیری از قضاوت‌های کاربر (حتی قضاوت‌های اولیه)، تخمین خود را از احتمالات بهبود بخشد و نتایج دقیق‌تری ارائه دهد.
- با **Okapi BM25** آشنا شدیم؛ یکی از موفق‌ترین و پرکاربردترین مدل‌های وزن‌دهی که از دل نظریه احتمالات بیرون آمده است. BM25 با فراتر رفتن از مدل باینری و در نظر گرفتن فراوانت کلمه و طول سند، عملکردی فوق‌العاده در دنیای واقعی از خود نشان داده و به یک استاندارد صنعتی تبدیل شده است.
- در نهایت، به نگاهی به مدل‌های پیشرفته‌تر مانند **شبکه‌های بیزین** انداختیم که امکان مدل‌سازی وابستگی‌های پیچیده بین کلمات را فراهم می‌کنند و پتانسیل ساخت سیستم‌های بسیار پیچیده‌تر را دارند.

بیاید این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

♦ **اصل رتبه‌بندی احتمالی (PRP) در موتورهای جستجوی مدرن:** وقتی شما در گوگل عبارت «رستوران ایتالیایی نزدیک من» را جستجو می‌کنید، موتور جستجو فقط به دنبال صفحاتی نیست که این کلمات را داشته باشند. به طور ضمنی، برای میلیون‌ها صفحه وب یک امتیاز احتمالی محاسبه می‌کند. صفحه‌ای که عبارت «رستوران ایتالیایی» دارد، نظرات مثبت دریافت کرده، در نقشه گوگل ثبت شده و اکنون باز است، امتیاز $P(R=1|d,q)$ بسیار بالاتری نسبت به صفحه‌ای خواهد داشت که فقط یک بار در یک مقاله گردشگری به کلمه «ایتالیایی» اشاره کرده است. این PRP در عمل است که نتایج را هوشمندانه مرتب می‌کند.

♦ **بازخورد مرتبط بودن، تکامل یافته و خودکار:** تصور کنید کاربری عبارت «یادگیری ماشین» را جستجو می‌کند. نتایج اولیه عمومی هستند. کاربر روی لینکی کلیک می‌کند که به «یادگیری عمیق برای بینایی کامپیوتر» اشاره دارد. سیستم این کلیک (بازخورد ضمنی) را ثبت می‌کند. در جستجوهای بعدی یا حتی در همان جلسه، سیستم ممکن است به طور خودکار نتایج بیشتری در حوزه بینایی کامپیوتر نمایش دهد. این نسخه مدرن و خودکار **ized** حلقه بازخورد احتمالی است؛ سیستمی که در حال یادگیری از رفتار کاربر است.

♦ **برتری BM25 بر مدل‌های ساده‌تر:** فرض کنید در حال جستجوی یک مقاله علمی طولانی هستید. یک مدل باینری (مانند BIM) سندی که یک بار عبارت «مدل احتمالی» را آورده است، با سندی که ۵۰ بار در جزئیات به آن پرداخته است، برابر در نظر می‌گیرد. اما BM25 به سند دوم امتیاز بسیار بالاتری می‌دهد، زیرا **فراوانت کلمه (tf)** را در نظر می‌گیرد. همچنین BM25 با نرمال‌سازی طول سند، درک می‌کند که یک کتاب ۵۰۰ صفحه‌ای که یک بار

کلمه را ذکر کرده، به اندازه یک مقاله ۵ صفحه‌ای که ۱۰ بار به آن اشاره کرده، «متمرکز» بر آن موضوع نیست. این هوشمندی BM25 را بسیار همسو با شهود انسانی و مؤثرتر می‌کند.

این فصل نشان داد که چگونه ریاضیات احتمالات می‌تواند به یک ابزار عملی و قدرتمند برای درک و بهبود بازیابی اطلاعات تبدیل شود. این مفاهیم فقط یک فصل از یک کتاب نیستند، بلکه ستون فقرات بسیاری از نوآوری‌های امروزی هستند. در فصل‌های بعدی، خواهیم دید که این اصول چگونه به مدل‌های زبانی احتمالی (فصل ۱۲) و سپس به تکنیک‌های یادگیری ماشین برای رتبه‌بندی (Learning to Rank) منجر شدند؛ مسیری که ما را به سمت ساخت سیستم‌های جستجوی هوشمندتر می‌برد که نه تنها کلمات را می‌فهمند، بلکه به نیاز پنهان و نیت کاربر نیز نزدیک‌تر می‌شوند.